

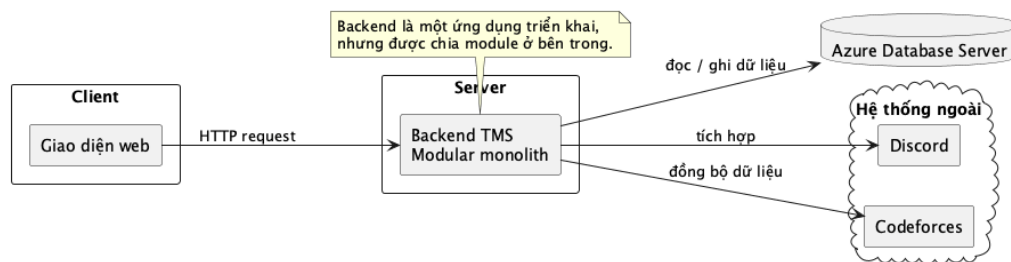
I. Kiến trúc tổng thể hệ thống

Phần này trình bày kiến trúc tổng thể của hệ thống Tutor Management System (TMS) ở mức thiết kế hệ thống. Nội dung tập trung vào các thành phần chính, cách bố trí các thành phần và quan hệ phụ thuộc giữa chúng. Các luồng nghiệp vụ chi tiết không được lặp lại trong phần này vì đã được mô hình hóa ở phần yêu cầu chức năng.

1. Kiểu kiến trúc được áp dụng

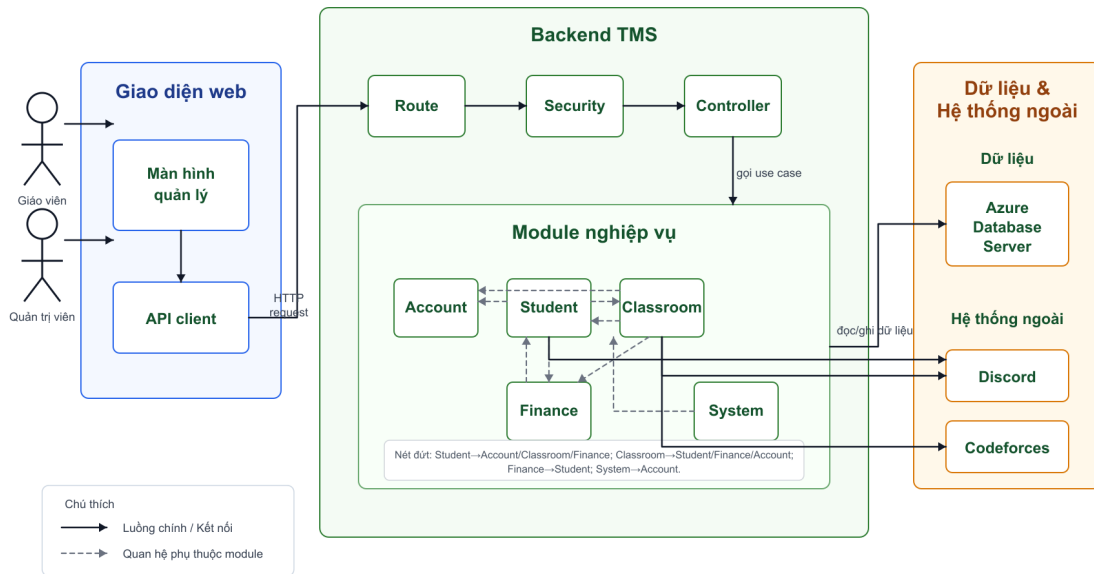
TMS sử dụng mô hình client-server: trình duyệt gửi request đến backend, backend xử lý nghiệp vụ và lưu dữ liệu trên Azure Database Server.

Ở mức triển khai backend, hệ thống chọn kiến trúc modular monolith. Backend vẫn là một ứng dụng duy nhất, nhưng được chia theo các ranh giới nghiệp vụ bên trong. Cách này đủ rõ ràng cho phạm vi đề án và tránh độ phức tạp của nhiều dịch vụ độc lập.



Hình 1: Kiểu kiến trúc được áp dụng trong hệ thống TMS

2. Các thành phần chính



Hình 2: Các thành phần chính và quan hệ phụ thuộc

Các thành phần chính được chia thành bốn nhóm để thể hiện rõ trách nhiệm và ranh giới của từng phần trong hệ thống.

Nhóm giao diện

- **Giao diện web:** cung cấp màn hình thao tác cho giáo viên và quản trị viên.
- **API client:** đóng gói việc gửi HTTP request từ giao diện đến các endpoint backend.

Nhóm backend

- **Route:** nhận request, kiểm tra định dạng dữ liệu đầu vào và chuyển request vào controller tương ứng.
- **Controller:** điều phối request đã hợp lệ thành lời gọi use case, chuẩn hóa response trả về cho giao diện.
- **Security:** xác thực token, kiểm tra vai trò và kiểm tra phạm vi sở hữu dữ liệu.
- **Account module:** quản lý đăng nhập, hồ sơ, tài khoản giáo viên và định danh liên quan tài khoản.
- **Student module:** quản lý học sinh, ghi danh, chuyển lớp, rút khỏi lớp, lưu trữ và dữ liệu học sinh liên quan Discord.

- **Classroom module:** quản lý lớp học, lịch học, buổi học, điểm danh, Discord của lớp, thông báo và Gym.
- **Finance module:** quản lý giao dịch, khoản phí, số dư học sinh và báo cáo tài chính.
- **System module:** quản lý tài khoản giáo viên và cấu hình hệ thống dùng chung.

Nhóm dữ liệu

- **Azure Database Server:** lưu dữ liệu nghiệp vụ và dữ liệu đã đồng bộ từ hệ thống ngoài.

Nhóm hệ thống ngoài

- **Discord:** hỗ trợ định danh, guild, channel, tin nhắn và các chức năng lớp học liên quan Discord.
- **Codeforces:** hỗ trợ dữ liệu Gym, bài tập và bảng xếp hạng luyện tập.

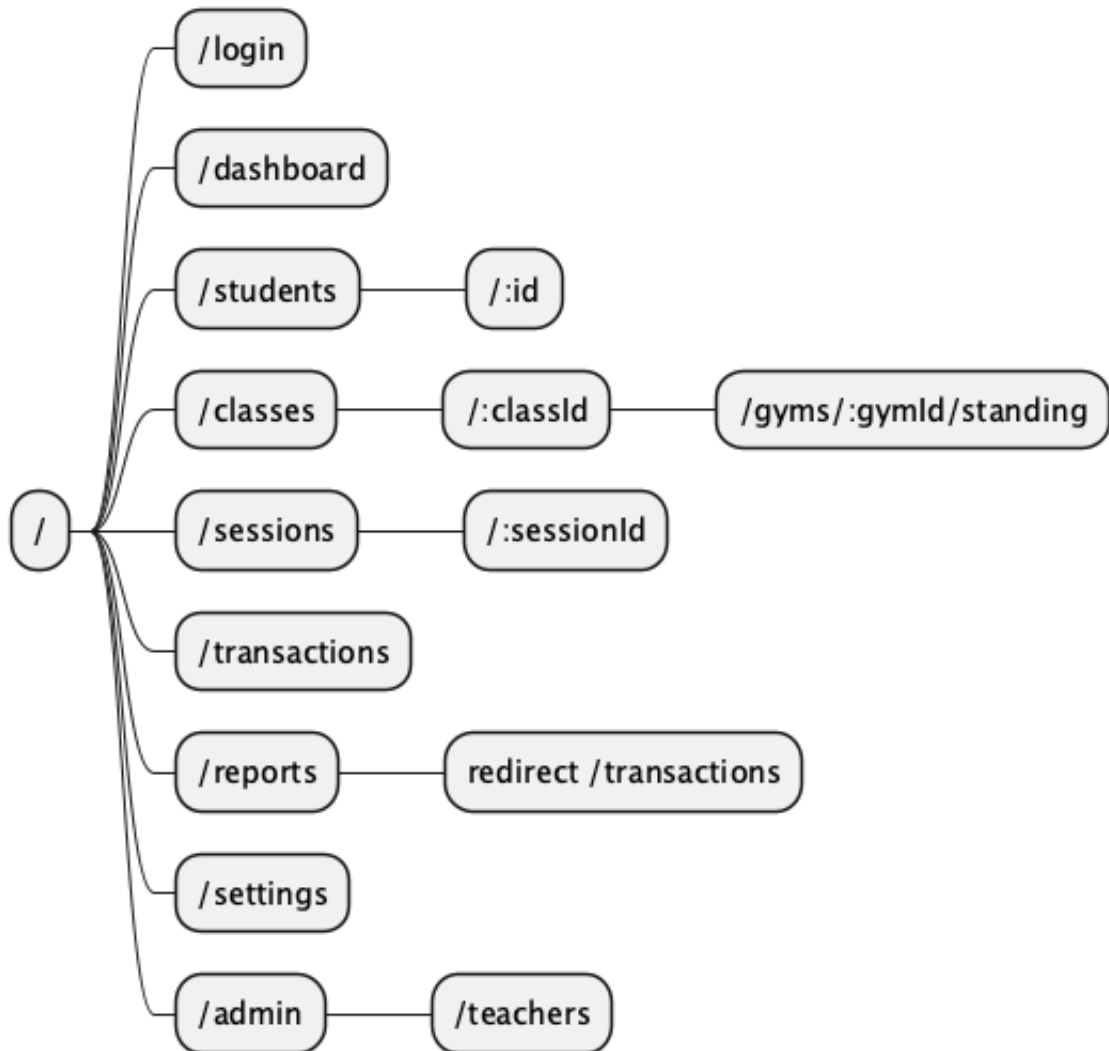
Backend được chia theo ranh giới nghiệp vụ. Bảng dưới đây tóm tắt phạm vi của các module chính.

Module	Phạm vi trách nhiệm
Account	Đăng nhập, hồ sơ người dùng, tài khoản giáo viên và các định danh liên quan tài khoản.
Student	Học sinh, ghi danh, chuyển lớp, rút khỏi lớp, lưu trữ học sinh và dữ liệu học sinh liên quan Discord.
Classroom	Lớp học, lịch học, buổi học, điểm danh, thiết lập Discord cho lớp, thông báo và Gym.
Finance	Giao dịch tài chính, khoản phí, số dư học sinh và báo cáo tài chính.
System	Chức năng quản trị hệ thống như quản lý tài khoản giáo viên và cấu hình bot Discord.
Security	Xác thực, phân quyền và kiểm tra phạm vi truy cập. Đây là lớp dùng chung, không phải module nghiệp vụ độc lập.

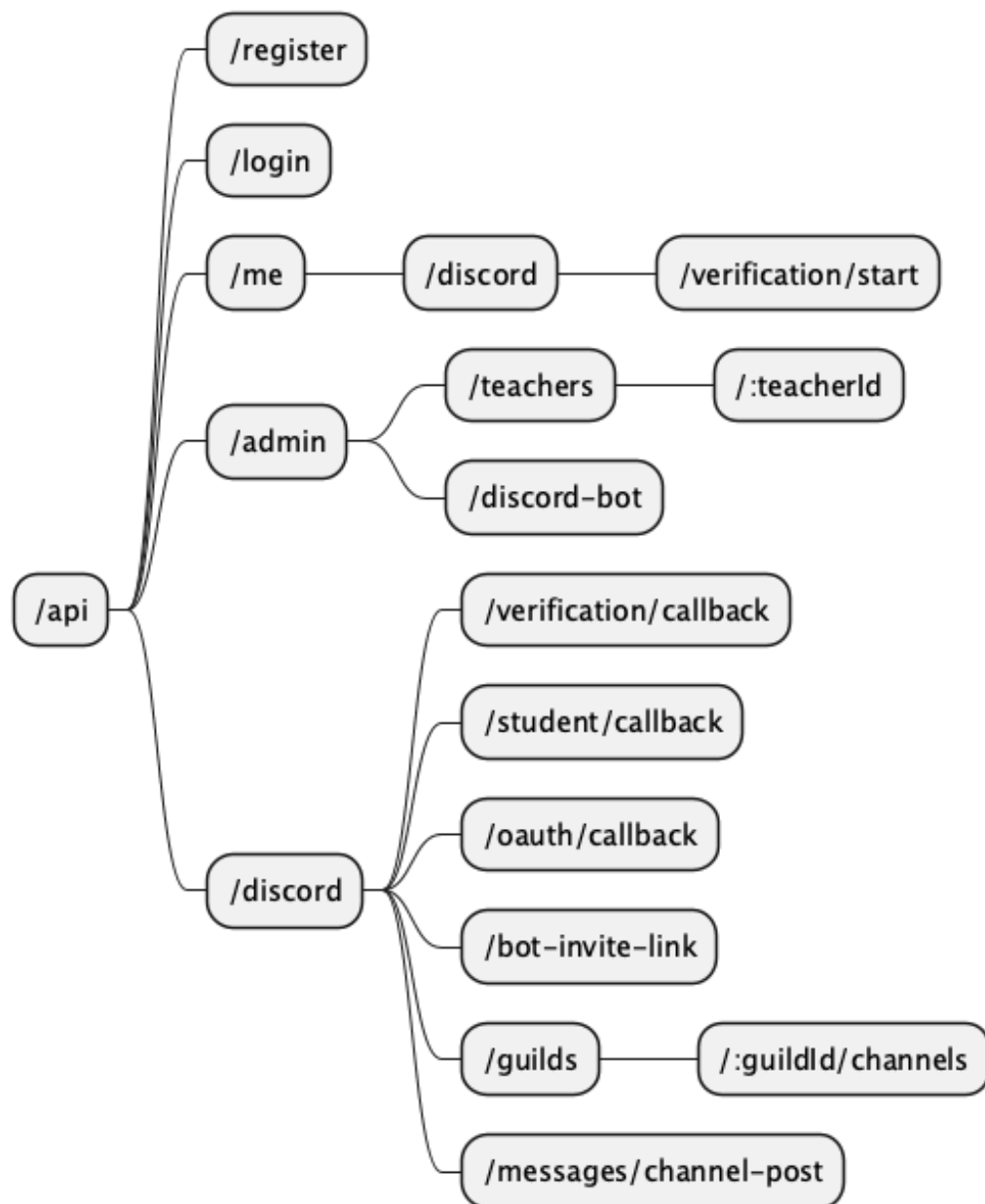
Các module backend chạy trong cùng một ứng dụng. Khi cần dữ liệu hoặc hành vi từ module khác, hệ thống ưu tiên điểm giao tiếp công khai thay vì phụ thuộc vào chi tiết nội bộ của module đó.

3. Thiết kế API endpoint

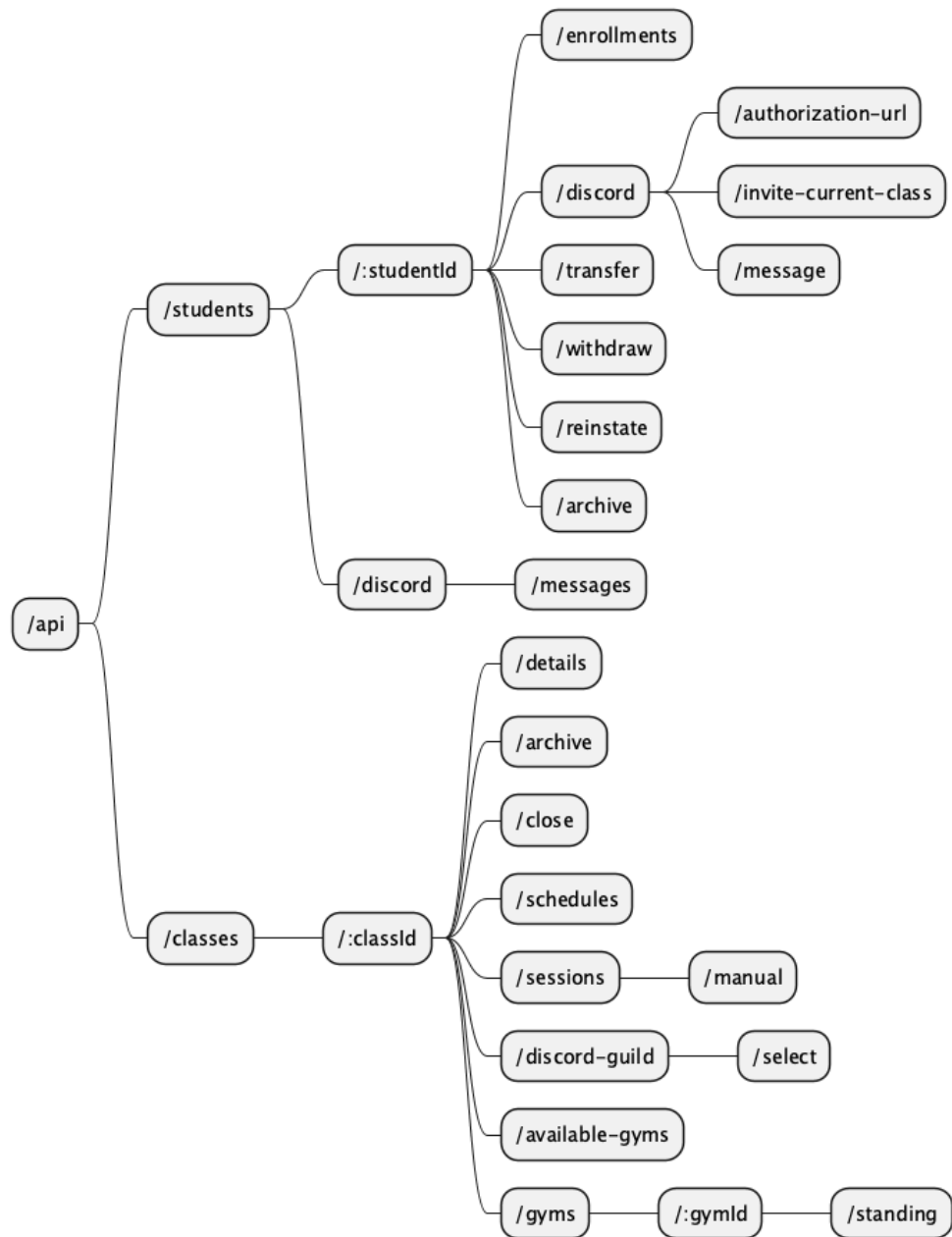
Hệ thống sử dụng REST API.



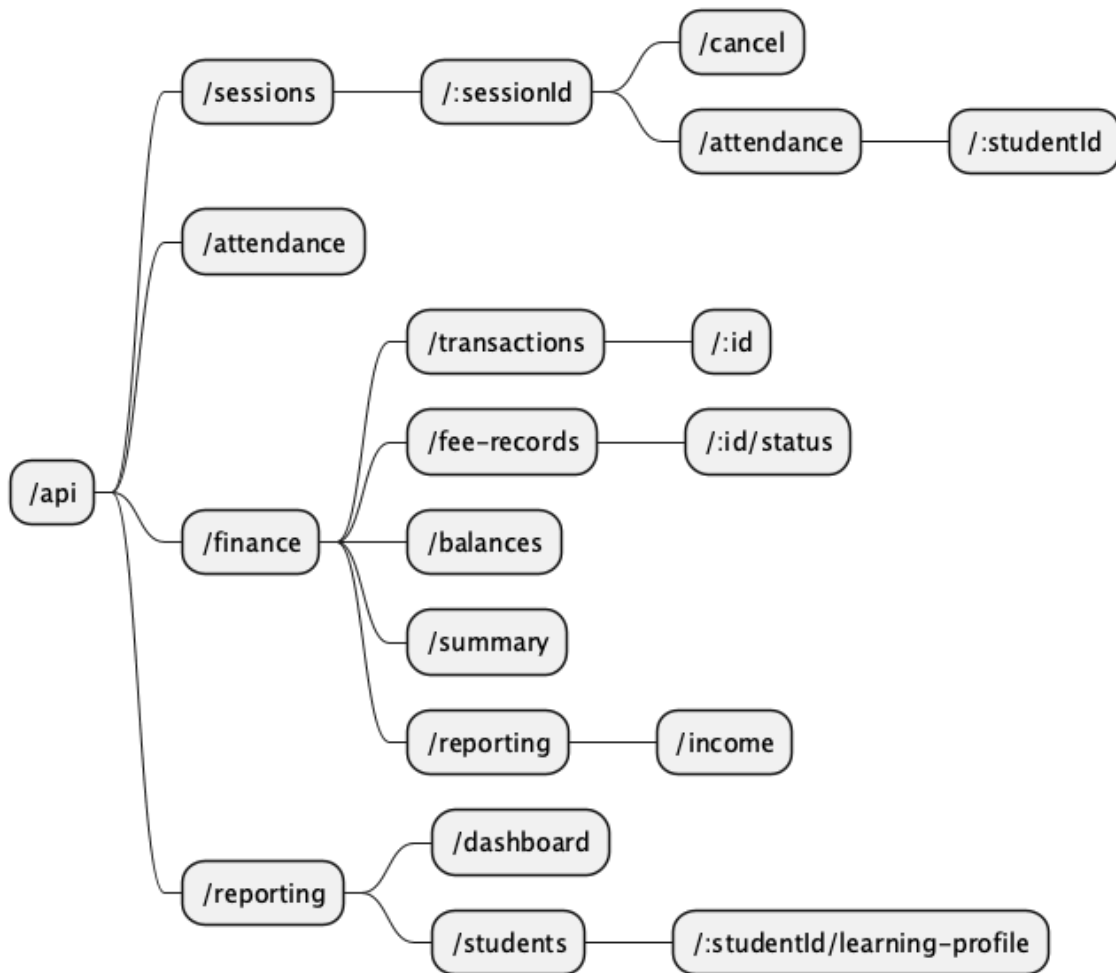
Hình 3: Các endpoint giao diện hiện tại của frontend



Hình 4: Các REST endpoint backend: tài khoản, quản trị và Discord

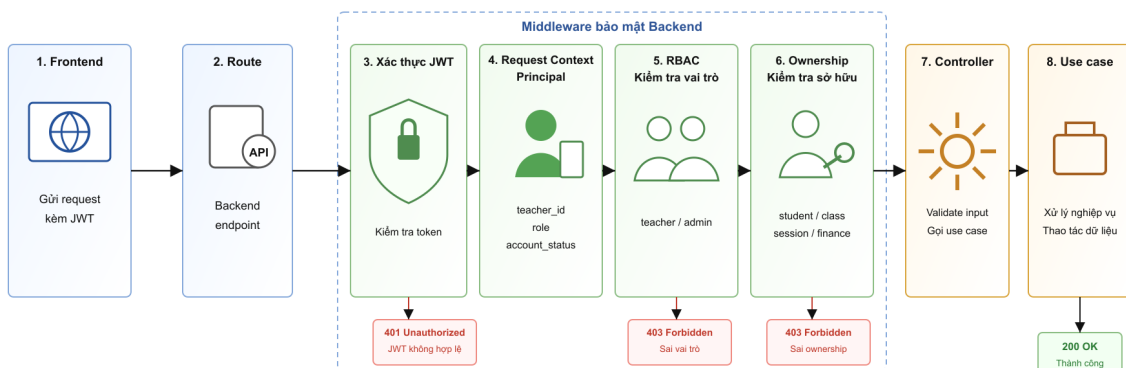


Hình 5: Các REST endpoint backend: học sinh và lớp học



Hình 6: Các REST endpoint backend: buổi học, điểm danh, tài chính và báo cáo

4. Bảo mật và kiểm soát truy cập



Hình 7: Luồng kiểm soát truy cập trước khi xử lý nghiệp vụ

Bảo mật được đặt ở lớp middleware của backend. Request từ frontend đi vào route backend, sau đó phải lần lượt qua các bước xác thực JWT, tạo request context, kiểm tra vai trò bằng RBAC và kiểm tra quyền sở hữu dữ liệu trước khi đến controller.

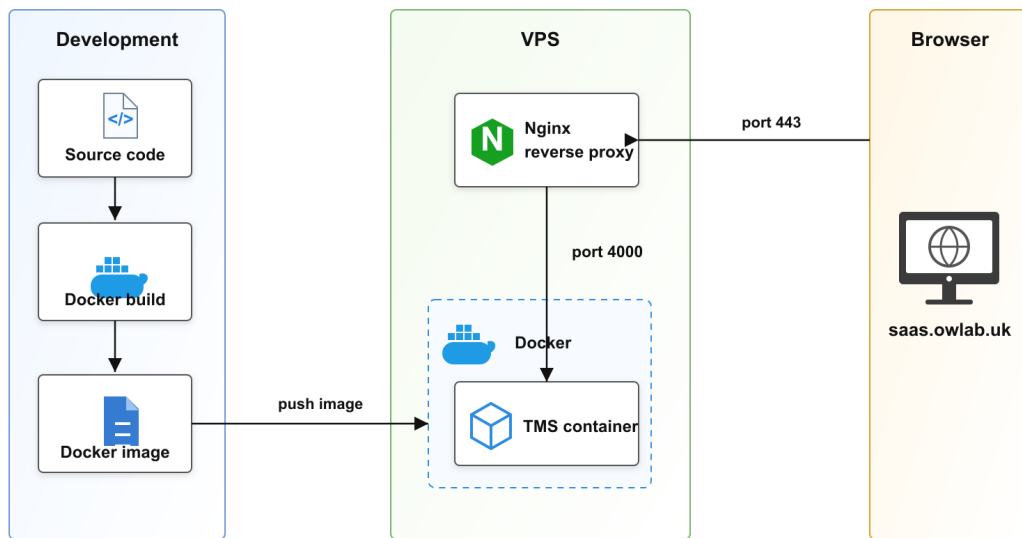
Quy tắc kiểm soát truy cập

- Mọi request cần đăng nhập phải gửi JWT trong header **Authorization:Bearer<token>**.
- JWT authentication kiểm tra chữ ký, thời hạn và định dạng token. Nếu token thiếu hoặc không hợp lệ, request bị chặn với **401Unauthorized**.
- Sau khi JWT hợp lệ, backend tạo principal trong request context, gồm **teacher_id**, **role** và trạng thái tài khoản.
- RBAC kiểm tra vai trò có được phép đi vào route hay không. Admin được dùng route quản trị; teacher được dùng route nghiệp vụ của giáo viên. Sai vai trò bị chặn với **403Forbidden**.
- Ownership check bảo đảm teacher chỉ truy cập dữ liệu thuộc phạm vi sở hữu của mình, gồm học sinh, lớp học, buổi học, hồ sơ tài chính và Gym được phân công. Sai phạm vi sở hữu cũng bị chặn với **403Forbidden**.

Mô tả luồng

1. Frontend gửi HTTP request kèm JWT đến backend route.
2. Route chuyển request qua middleware bảo mật.
3. Middleware xác thực JWT và trích xuất principal.
4. Hệ thống kiểm tra vai trò bằng RBAC.
5. Với route truy cập dữ liệu theo giáo viên, hệ thống kiểm tra ownership.
6. Request hợp lệ mới được chuyển đến controller và use case nghiệp vụ.
7. Kết quả xử lý được trả về cho frontend.

5. Sơ đồ triển khai



Hình 8: Sơ đồ triển khai phần mềm

1. Mã nguồn ở môi trường Development được build thành Docker image.
2. Docker image được push từ môi trường Development sang VPS.
3. Trên VPS, Docker chạy image này thành TMS container.
4. Người dùng mở Browser và truy cập domain **saas.owlab.uk**.
5. Request từ Browser đi vào VPS qua port 443.
6. Nginx trên VPS nhận request ở port 443 và đóng vai trò reverse proxy.
7. Nginx chuyển tiếp request vào TMS container qua port 4000.
8. TMS container xử lý request và trả kết quả ngược lại qua cùng đường đi.